ElectroSim-DunnECASA Suite

A publication-ready toolkit for electrochemical cyclic voltammetry (CV) analysis: Dunn's Method, ECASA, and Randles–Sevcik.

Maintained by **Rajeev Kumar** (rkumar@nccu.edu), North Carolina Central University.

> Single canonical README for the whole project — end-user, web-demo, sample-data, build, and installer docs are all here.

What it does

Analysis	Output
Dunn's Method	b-value charge-storage classification (capacitive / battery / pseudocapacitor / hybrid) + capacitive vs diffusion current decomposition
ECASA	Electrochemically active surface area from non-Faradaic double-layer capacitance
Randles–Sevcik	Diffusion coefficient D from peak current vs √(scan rate)

Available in **three forms**, all with the same analyses:

1. **Hosted web demo (Streamlit)** — fastest way in, nothing to install: **electrosim-dunnecasa-suite.streamlit.app**
2. **Web app (PyWebView desktop window)** — the same Streamlit app in a native window, fully offline.
3. **Tkinter desktop GUI** — lightweight native fallback.

Getting started

Web demo: open **https://electrosim-dunnecasa-suite.streamlit.app/**, upload a CV file (wide Excel or stacked CSV), get all three analyses instantly.

Desktop install (Windows 64-bit): download **ElectroSim-DunnECASA-Suite-Setup-2.0.exe** (163 MB) from the Releases page → double-click → 3 screens (Welcome → Installing → Finish, ~15 s, no admin rights). Inside the app, click **Load files...** → pick a template from `sample_data/` → switch to the analysis tab you need.

> New / non-technical? See **QUICK_START.md** for a 1-page plain-English walkthrough (including the SmartScreen prompt). Then docs/user_guide.md for the full walkthrough and docs/methods.md for the scientific background.

Verifying the download

`Setup-2.0.exe` is unsigned, so Windows SmartScreen shows "Windows protected your PC" the first time — click **More info** → **Run anyway**. To confirm the file is genuine, check its SHA-256:

```
Get-FileHash -Algorithm SHA256 $env:USERPROFILE\Downloads\ElectroSim-DunnECASA-Suite-Setup-2.0.
```

Expected (case-insensitive): `7DC6F05D6EA99ACB5E6551FB862064BB6C5B80D4B10EFE7CF65E949C91DF7B99`

System requirements

- Windows 10 1809 (build 17763) or later, 64-bit
- ~700 MB free disk space
- Edge WebView2 (preinstalled on Windows 10/11; required only for the hybrid build — the Tkinter fallback works without it)

> The source code (`.py`) is intentionally not published; the suite is distributed in binary form only. See LICENSE and NOTICE.

Input data formats

Format	Layout
Wide Excel	Header-encoded scan rate, one column pair per rate (e.g. `Potential at 50 mV/s` , `Current at 50 mV/s`)
Stacked CSV / Excel	Three columns: Potential, Current, ScanRate
Single CV	Two columns: Potential, Current (Dunn / ECASA / RS need multiple scan rates)

Sample templates

Ready-to-load templates live in `release/sample_data/` — open any from the **Data Files** tab:

File	Purpose
`Dunn method template.xlsx`	Dunn's Method, wide Excel with annotated two-row header
`ECASA CV template.xlsx`	ECASA non-Faradaic window, wide Excel two-row header

> **Using a template?** Paste your data in and upload as-is. **Do not modify the first two rows** — row 1 (scan-rate headers) and row 2 (Potential / Current sub-headers) are required. Add or remove data *rows* freely.

Recommended workflow: open `Dunn method template.xlsx` in **Data Files** → **Data Plotting** → **Plot** (confirm the CV looks right) → **Dunn's Method** → set branch and prominence → **Run analysis**.

Running the web app locally

The web app lives in `release/web-demo/`. It is a Streamlit app and **must** be launched with `streamlit run`, not plain `python`:

```
cd release\web-demo

# RIGHT — Streamlit boots its runtime, then exec's the loader
streamlit run electrosim_app.py

# RIGHT — pick a specific Python version (must match a shipped .pyc; see below)
py -3.12 -m streamlit run electrosim_app.py

# WRONG — plain python prints a friendly error and exits
python electrosim_app.py
```

Why `streamlit run`? The engine calls `st.set_page_config`, `st.sidebar.*`, etc. at module-load time, which need Streamlit's per-thread `ScriptRunContext`. The loader detects a missing context and prints a clear "WRONG / RIGHT" message instead of an internals traceback.

Pick the right Python — one that has **both** Streamlit **and** a matching engine `.pyc` (currently `cp312` for 3.12, `cp314` for 3.14). If deps are missing:

```
py -3.14 -m pip install -r release\web-demo\requirements.txt
```

> ⚠️ A running Streamlit server loads the engine bytecode **once at startup** and does not hot-reload a recompiled `.pyc` — **restart the server** after any engine change.

How the web app is packaged (bytecode-only)

The Python in `release/web-demo/` is **compiled bytecode**:

- `electrosim_app.py` — small Streamlit entry-point loader (the only `.py`, no analysis logic). This is what Streamlit Cloud runs.

- `electrosim_engine.cp312.pyc` / `electrosim_engine.cp314.pyc` — the analysis engine, one per supported Python version. The loader auto-picks the match for the running interpreter and `exec()`s it.

> **Python-version pinning.** A `.pyc` is keyed to the exact interpreter version. If Streamlit Cloud upgrades and the demo fails with a "magic number" error, recompile upstream source (`py -3.XX -m py_compile streamlined_app.py`) and drop `__pycache__/*.cpython-3XX.pyc` in as `electrosim_engine.cp3XX.pyc` .

Citation

If you use this in academic work, please cite via the metadata in CITATION.cff. A DOI is issued for each release through Zenodo.

License & permitted use

- **Permitted:** personal use (desktop or hosted demo) for evaluation and learning; academic / non-commercial research, including publications, provided the Suite is cited.
- **Not permitted without prior written permission:** commercial use of the app, demo, or their outputs; re-deployment under any other name or hosting; decompilation, disassembly, or reverse-engineering of the bytecode.

 Contact: rkumar@nccu.edu.

For maintainers — building from source

> End users don't need this — they download the prebuilt installer. The Python source (`streamlined_app.py` , `electrosim_launcher.py` , etc.) is **not** in this public repo; the maintainer builds from a private tree. Building in a clone of the public repo fails with "module not found" — expected.

Build reference files (tracked in `release/build/`):

File	What it is
`ElectroSim-DunnECASA-Suite.spec`	PyInstaller config for the **PyWebView/Streamlit hybrid** build (main app)
`ElectroSim-DunnECASA-Suite-Tkinter.spec`	PyInstaller config for the **Tkinter** fallback build
`electrosim_build.ps1`	PowerShell wrapper running PyInstaller against both specs (`-Target streamlit|tkinter|all` , optional `-Clean`)

Full release workflow (from the private source tree at the project root):

```
# 1. Build both PyInstaller bundles -> dist\ at the project root
.\release\build\electrosim_build.ps1 -Clean

# 2. Install Inno Setup 6 (once): winget install --id JRSoftware.InnoSetup
# 3. Compile the installer
& "${env:LocalAppData}\Programs\Inno Setup 6\ISCC.exe" release\installer\electrosim_setup.iss
# -> release\installer\Output\ElectroSim-DunnECASA-Suite-Setup-X.Y.exe  (upload to GitHub Relea
```

What the installer produces: a 3-screen, no-UAC, per-user install into `%LocalAppData%\Programs\ElectroSim-DunnECASA Suite\` with a Desktop shortcut (hybrid build), a Start Menu folder (main app, User Guide, Sample Data, Tkinter fallback + Uninstall under `Tools\`), and `LICENSE` / `NOTICE` / `README.md` / `CHANGELOG.md` / `CITATION.cff` / `sample_data/` / `docs/` at the install root. The Tkinter build is a fallback for when the hybrid can't launch (e.g. broken Edge WebView2).

Cutting a new release (2.1, 3.0, ...):

1. Bump `MyAppVersion` in `release/installer/electrosim_setup.iss` .
2. Update CHANGELOG.md (shown on the post-install screen).
3. **Do NOT change** `AppId` — its GUID is what makes Windows upgrade in place instead of side-by-side.
4. Rebuild bundles, recompile the `.iss` , upload the new `Setup-X.Y.exe` .
5. **Recompute the SHA-256** of the new `.exe` and update it in this README and in QUICK_START.md.
6. **Recommended:** submit the new unsigned `Setup.exe` to Microsoft so SmartScreen/Defender clears it (24–72 h; re-submit per release since the hash changes).

Build troubleshooting: `ISCC error: Source file not found: ..\..\dist\…` → run PyInstaller first / run ISCC from the project root. `ISCC: command not found` → Inno Setup not installed or not on PATH. Installer crashes on a clean VM → incomplete PyInstaller bundle (run the `.exe` from `dist\…\` directly to confirm). Two Add/Remove Programs entries after upgrade → someone changed `AppId` .

Support / bug reports

Open an Issue. Please include: the exact build name from **Help** → **About**, the input file format (anonymise data if needed), and a screenshot of the error if applicable.